

Actividad 3

Fernando Israel Rios Garcia-AL02877016@tecmlenio.mx

Artur Mikhail Lazurenko Mannanov-AL03028732@tecmlenio.mx

Angel Cuevas Lopez-AL03033114@tecmlenio.mx

Universidad Tecmlenio

LSTI2308: Sistemas Operativos

Ana Cruz

15 de febrero de 2026

Índice

1. Introducción	4
2. Parte 1: Análisis de la administración de la memoria	4
2.1. Propuesta Técnica: Implementación de Segmentación	4
2.1.1. Introducción	4
2.1.2. Modelo de Gestión de Memoria	4
2.1.3. Análisis de Etapas Operativas	4
2.1.4. Protocolo de Recuperación del Sistema	6
2.1.5. Referencias Bibliográficas	6
3. Parte 2: Diseño y organización del sistema de archivos	6
3.1. Exploración práctica	6
3.1.1. Comandos utilizados	6
3.1.2. Directorios importantes del sistema Linux	7
3.2. Actividad de diseño: Servidor CMS	7
3.2.1. Diagrama de estructura propuesta	7
3.2.2. Organización de archivos y directorios	7
3.2.3. Configuración de permisos y seguridad	8
4. Parte 3: Simulación de algoritmos de planificación del disco	8
4.1. Resultados de la simulación	8
4.1.1. FCFS (First-Come, First-Served)	8
4.1.2. SSTF (Shortest Seek Time First)	8
4.1.3. SCAN (Elevator Algorithm)	9
4.2. Análisis	9
5. Parte 4: Configuración y análisis de RAID en Linux	9
5.1. Práctica en Linux: Configuración de RAID	9

5.1.1. RAID 0 (Striping)	10
5.1.2. RAID 1 (Mirroring)	10
5.2. Análisis comparativo	11
6. Parte 5: Informe final	11

Índice de figuras

1. Diagrama de Segmentación de Memoria	5
2. Gráfico de movimiento FCFS	9
3. Gráfico de movimiento SSTF	10
4. Gráfico de movimiento SCAN	11

1. Introducción

Este documento presenta el desarrollo de la Actividad 3, abordando temas clave en sistemas operativos como la administración de memoria, el diseño de sistemas de archivos, algoritmos de planificación de disco y almacenamiento RAID. Se incluyen propuestas técnicas, análisis prácticos y simulaciones para demostrar la comprensión de estos conceptos.

2. Parte 1: Análisis de la administración de la memoria

2.1. Propuesta Técnica: Implementación de Segmentación

2.1.1. *Introducción*

Esta propuesta define el mecanismo de gestión de memoria principal para un sistema operativo diseñado para entornos de alta eficiencia. Se selecciona la segmentación como esquema fundamental, permitiendo que el sistema gestione la memoria basándose en la visión lógica del programador y la estructura natural de los procesos.

2.1.2. *Modelo de Gestión de Memoria*

A diferencia de sistemas operativos robustos y complejos (monolíticos tradicionales), este núcleo se orienta a la eficiencia mediante tareas particulares y una estructura simplificada.

Eficiencia de Segmentos: El sistema es veloz en el uso de memoria debido a una segmentación correcta de los programas. El cargador del SO respeta la división lógica (código, datos, pila) establecida durante la compilación.

Estrategia de Asignación: Se requiere un análisis previo del espacio necesario para cada segmento. El núcleo utiliza esta información para minimizar el desperdicio y optimizar el tiempo de respuesta del gestor de memoria.

2.1.3. *Análisis de Etapas Operativas*

Eta 1: Planificación de la Tabla de Segmentos. El sistema operativo establece una Tabla de Segmentos para cada proceso. El análisis previo permite definir la

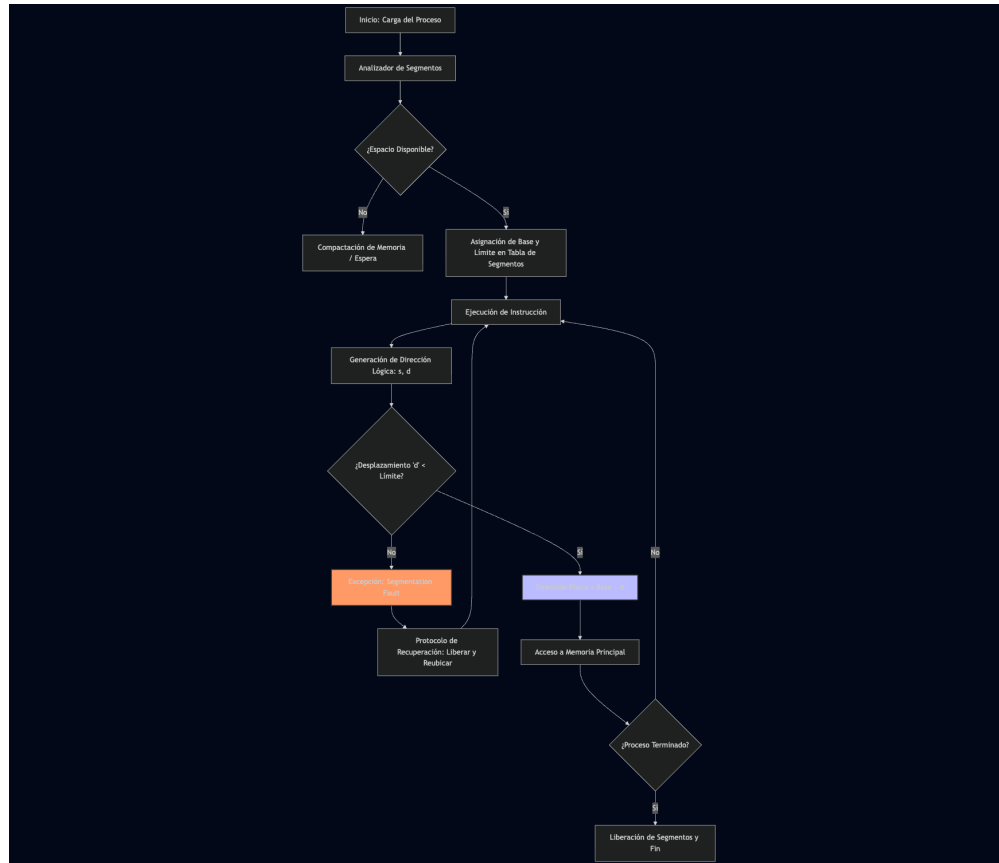


Figura 1

Diagrama de Segmentación de Memoria

base y el límite de cada entrada, asegurando que la protección de memoria sea estricta y el aprovechamiento del espacio sea máximo.

Etap 2: Traducción y Ejecución. Durante la ejecución de procesos, la unidad de gestión de memoria utiliza la segmentación para acceder a instrucciones de forma inmediata. Al ser un sistema diseñado para tareas particulares, se evitan las latencias de estructuras de paginación multinivel, favoreciendo la velocidad bruta.

Etap 3: Control de Restricciones y Excepciones. Dada la naturaleza de la segmentación en un entorno de memoria limitada:

- **Fragmentación Externa:** El núcleo gestiona la fragmentación externa derivada de la asignación de bloques de tamaño variable.

- **Gestión de Violación de Límites:** Cuando un cálculo de dirección lógica excede el límite definido en la partición del segmento, el hardware genera una excepción de segmentación (Segmentation Fault).

2.1.4. Protocolo de Recuperación del Sistema

Ante el fallo de un proceso por exceso de límite de segmento, el núcleo ejecuta el siguiente protocolo de recuperación:

1. Interrupción del proceso mediante excepción de protección.
2. Liberación de los marcos de memoria comprometidos.
3. Recarga de los segmentos en la memoria principal (reubicación si es necesario).
4. Reinicio de la ejecución del proceso para restaurar el servicio.

2.1.5. Referencias Bibliográficas

Para la elaboración de esta propuesta se consultaron las siguientes fuentes Análisis Multimedia, s.f.; Grokipedia, s.f.-a, s.f.-b; Universidad de Almería, s.f.

3. Parte 2: Diseño y organización del sistema de archivos

3.1. Exploración práctica

En esta actividad se exploraron comandos y directorios clave en un entorno Linux.

3.1.1. Comandos utilizados

- **pwd:** Muestra la ruta del directorio actual.
- **ls:** Lista el contenido de directorios. `ls -l` para detalles y permisos, `ls -a` para archivos ocultos.
- **cd:** Cambiar de directorio (e.g., `cd /`, `cd /home`).
- **tree:** Ver estructura jerárquica (requiere instalación: `sudo apt install tree`).

3.1.2. Directorios importantes del sistema Linux

1. / (**root**): Raíz del sistema de archivos, inicio de toda la jerarquía.
2. /**home**: Archivos personales de los usuarios.
3. /**etc**: Archivos de configuración del sistema (red, usuarios, servicios).
4. /**var**: Archivos variables como logs, caché, bases de datos temporales.
5. /**bin**: Programas esenciales y comandos básicos (**ls**, **cp**, **rm**).

3.2. Actividad de diseño: Servidor CMS

Objetivo: Crear un sistema que soporte múltiples sitios web, mantenga la separación de datos, sea seguro y fácil de administrar.

3.2.1. Diagrama de estructura propuesta

```

/
+-- cms
|   +-- sites
|       |   +-- sitio1 (public_html, config, logs, uploads, backups)
|       |   +-- sitio2 (public_html, config, logs, uploads, backups)
|       +-- shared (templates, plugins, libraries)
|   +-- database
|   +-- admin (scripts)

```

3.2.2. Organización de archivos y directorios

- /**cms/sites/**: Contiene espacios independientes para cada sitio web.
 - **public_html**: Archivos visibles (HTML, CSS, JS).
 - **config**: Configuración privada (conexión BD), no accesible públicamente.
 - **logs**, **uploads**, **backups**: Registros, subidas y copias de seguridad aisladas.

- **/cms/shared/**: Recursos compartidos para evitar duplicidad (plantillas, plugins).
- **/cms/database/**: Archivos de base de datos con acceso restringido.
- **/cms/admin/**: Scripts de mantenimiento y administración.

3.2.3. *Configuración de permisos y seguridad*

Se utiliza el sistema de permisos Linux (r, w, x) y usuarios específicos:

- **Usuarios/Grupos:** admin (administrador), web_sitio1, web_sitio2, grupo cms_sites.
- **Permisos recomendados:**
 - Sitios: chmod 750 (Dueño control total, Grupo acceso limitado).
 - public_html: chmod 755 (Lectura pública).
 - config, database: chmod 700 (Solo propietario).
 - shared: chmod 755 (Solo lectura para sitios).

Medidas de seguridad: Separación de datos por usuarios, restricción de permisos en archivos sensibles, copias de seguridad automáticas y registro de actividad en logs.

4. Parte 3: Simulación de algoritmos de planificación del disco

4.1. Resultados de la simulación

Se implementó una simulación para los algoritmos FCFS, SSTF y SCAN con la secuencia de solicitudes: 95, 180, 34, 119, 11, 123, 62, 64 (Inicio: 50).

4.1.1. *FCFS (First-Come, First-Served)*

Orden de atención: 50 → 95 → 180 → 34 → 119 → 11 → 123 → 62 → 64

Movimiento total: 644 pistas.

4.1.2. *SSTF (Shortest Seek Time First)*

Orden de atención: 50 → 62 → 64 → 34 → 11 → 95 → 119 → 123 → 180

Movimiento total: 236 pistas.

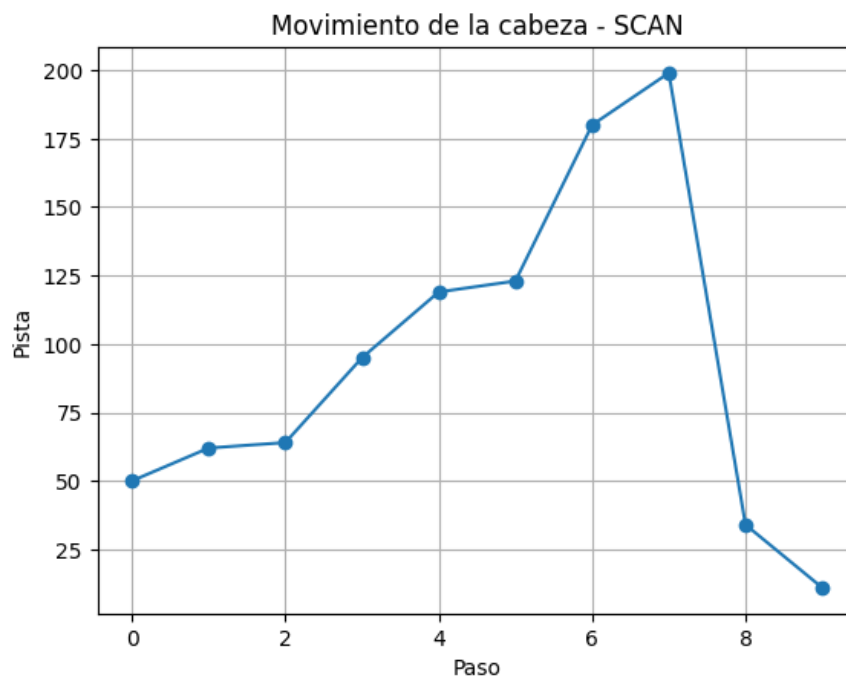


Figura 2

Gráfico de movimiento FCFS

4.1.3. SCAN (Elevator Algorithm)

Orden de atención (hacia la derecha): 50 → 62 → 64 → 95 → 119 → 123 → 180 → 199 → 34 → 11

Movimiento total: 337 pistas.

4.2. Análisis

El algoritmo **SSTF** resultó ser el más eficiente en este caso con un movimiento total de 236 pistas, minimizando el tiempo de búsqueda al atender siempre la solicitud más cercana. FCFS, aunque garantiza justicia, resultó en el mayor movimiento (644). SCAN ofrece un compromiso intermedio (337) y es más predecible que SSTF en cargas altas.

5. Parte 4: Configuración y análisis de RAID en Linux

5.1. Práctica en Linux: Configuración de RAID

Se utilizaron discos virtuales (`/dev/sdb`, `/dev/sdc`) y la herramienta `mdadm`.

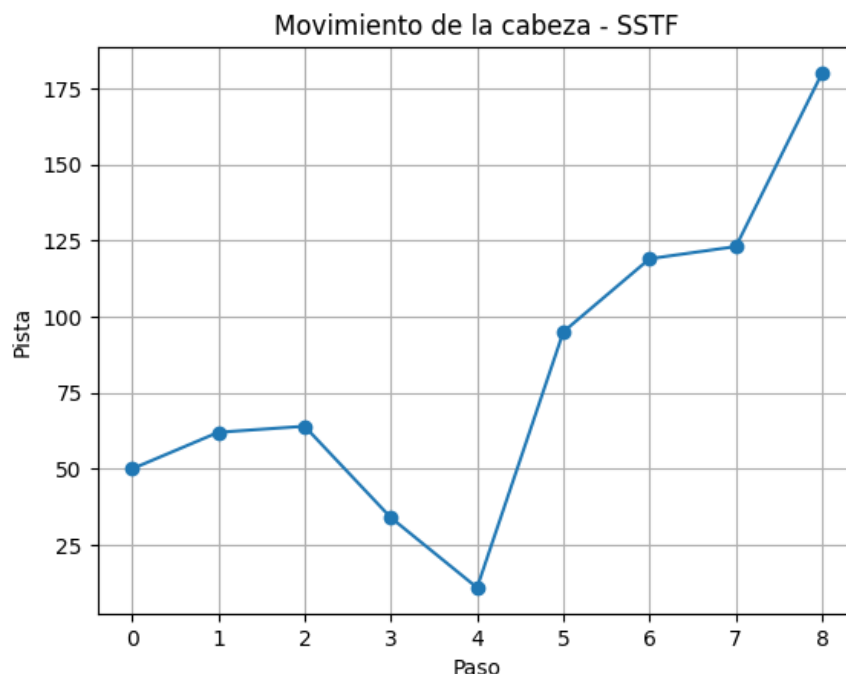


Figura 3

Gráfico de movimiento SSTF

5.1.1. RAID 0 (Striping)

- **Creación:** `sudo mdadm -create -verbose /dev/md0 -level=0 -raid-devices=2 /dev/sdb /dev/sdc`
- **Características:** Divide datos entre discos, ofreciendo mayor velocidad pero sin tolerancia a fallos.
- **Rendimiento (simulado):** Escritura ≈ 450 MB/s, Lectura ≈ 520 MB/s.

5.1.2. RAID 1 (Mirroring)

- **Creación:** `sudo mdadm -create -verbose /dev/md1 -level=1 -raid-devices=2 /dev/sdb /dev/sdc`
- **Características:** Crea una copia exacta (espejo). Alta tolerancia a fallos, menor velocidad de escritura.

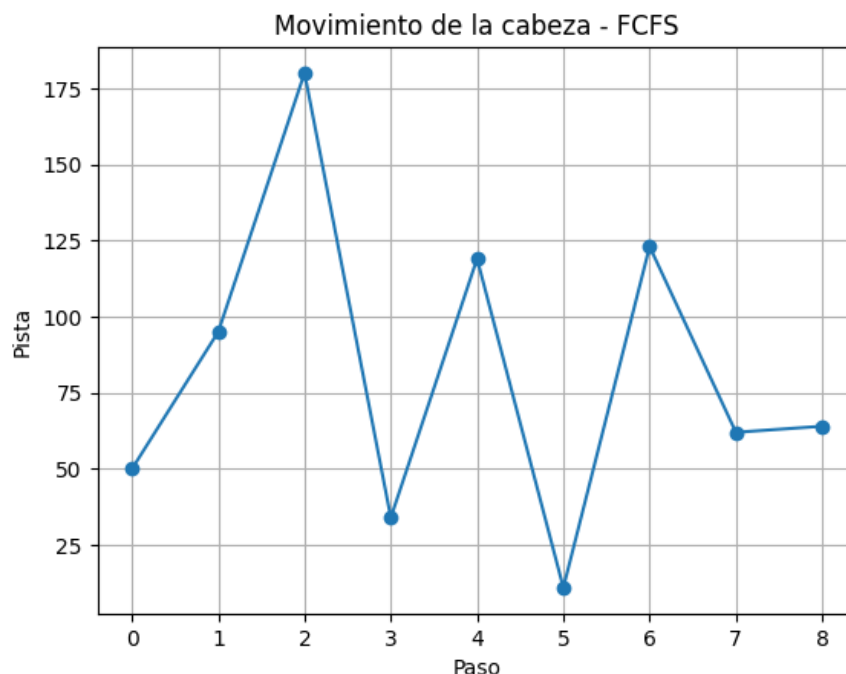


Figura 4

Gráfico de movimiento SCAN

- **Prueba de fallo:** Se simuló fallo en `/dev/sdb` (`-fail`, `-remove`) y el sistema continuó funcionando correctamente.
- **Rendimiento (simulado):** Escritura ≈ 210 MB/s, Lectura ≈ 240 MB/s.

5.2. Análisis comparativo

- **RAID 0:** Ideal para edición multimedia y datos temporales donde la velocidad es crítica. Riesgo alto de pérdida de datos.
- **RAID 1:** Recomendado para servidores web y bases de datos donde la integridad y disponibilidad son prioritarias.

6. Parte 5: Informe final

En esta actividad integradora se abarcaron aspectos esenciales de la administración de sistemas. Se diseñó una estrategia de segmentación para memoria embebida, priorizando la eficiencia. Se estructuró un sistema de archivos seguro para un CMS, aplicando permisos

granulares. Se demostró mediante simulación que algoritmos como SSTF optimizan el movimiento del disco frente a FCFS. Finalmente, la implementación de RAID corroboró las compensaciones teóricas entre rendimiento (RAID 0) y redundancia (RAID 1), permitiendo seleccionar la configuración adecuada según la criticidad de los datos.

Referencias

Análisis Multimedia. (s.f.). *Segmentación de Memoria y Ciclos de CPU*. Consultado el 13 de febrero de 2026, desde <https://www.youtube.com/watch?v=p9yZNLeOj4s>

Grokipedia. (s.f.-a). *Memory Paging Mechanisms*. Consultado el 13 de febrero de 2026, desde https://grokipedia.com/page/Memory_paging

Grokipedia. (s.f.-b). *Memory Segmentation Architecture*. Consultado el 13 de febrero de 2026, desde https://grokipedia.com/page/Memory_segmentation

Universidad de Almería. (s.f.). *Gestión de Memoria en Sistemas Operativos*. Consultado el 13 de febrero de 2026, desde <https://w3.ual.es/~rguirado/so/tema5.pdf>